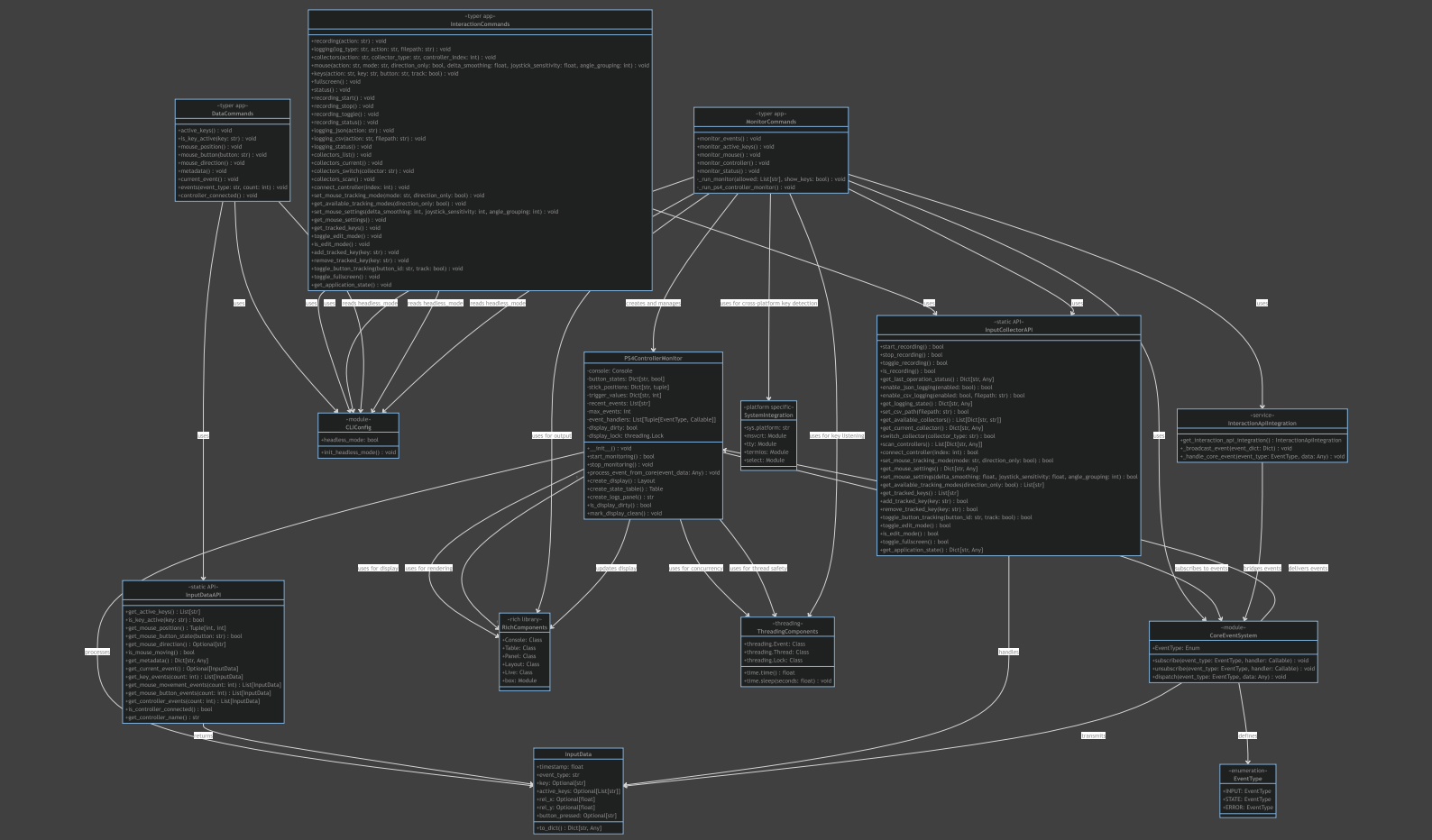

«typer app»
InteractionCommands
+recording(action: str) : void
+logging(log_type: str, action: str, filepath: str) : void
+collectors(action: str, collector_type: str, controller_index: int) : void
+mouse(action: str, mode: str, direction_only: bool, delta_smoothing: float, joystick_sensitivity: float, angle_grouping: int) : void
+keys(action: str, key: str, button: str, track: bool) : void
+fullscreen() : void
+status() : void
+recording_start() : void
+recording_stop() : void
+recording_toggle() : void
+recording_status() : void
+logging_json(action: str) : void
+logging_csv(action: str, filepath: str) : void
+logging_status() : void
+collectors_list() : void
+collectors_current() : void
+collectors_switch(collector: str) : void
+collectors_scan() : void
+connect_controller(index: int) : void
+set_mouse_tracking_mode(mode: str, direction_only: bool) : void
+get_available_tracking_modes(direction_only: bool) : void
+set_mouse_settings(delta_smoothing: int, joystick_sensitivity: int, angle_grouping: int) : void
+get_mouse_settings() : void
+get_tracked_keys() : void
+toggle_edit_mode() : void
+is_edit_mode() : void
+add_tracked_key(key: str) : void
+remove_tracked_key(key: str) : void
+toggle_button_tracking(button_id: str, track: bool) : void
+toggle_fullscreen() : void
+get_application_state() : void
«typer app»
DataCommands
+active_keys() : void
+is_key_active(key: str) : void
+mouse_position() : void
+mouse_button(button: str) : void
+mouse_direction() : void
+metadata() : void
+current_event() : void
+events(event_type: str, count: int) : void
+controller_connected() : void
«typer app»
MonitorCommands
+monitor_events() : void
+monitor_active_keys() : void
+monitor_mouse() : void
+monitor_controller() : void
+monitor_status() : void
_run_monitor(allowed: List[str], show_keys: bool) : void
_run_ps4_controller_monitor() : void
«module»
CLIConfig
-headless_mode: bool
+init_headless_mode() : void
PS4ControllerMonitor
-console: Console
-button_states: Dict[str, bool]
-stick_positions: Dict[str, tuple]
-trigger_values: Dict[str, int]
-recent_events: List[str]
-max_events: int
-event_handlers: List[Tuple[EventType, Callable]]
-display_dirty: bool
-display_lock: threading.Lock
+__init__() : void
+start_monitoring() : bool
+stop_monitoring() : void
+process_event_from_core(event_data: Any) : void
+create_display() : Layout
+create_state_table() : Table
+create_logs_panel() : str
+is_display_dirty() : bool
+mark_display_clean() : void
«platform specific»
SystemIntegration
+sys.platform: str
+msvcrt: Module
+tty: Module
+termios: Module
+select: Module
«static API»
InputCollectorAPI
+start_recording() : bool
+stop_recording() : bool
+toggle_recording() : bool
+is_recording() : bool
+get_last_operation_status() : Dict[str, Any]
+enable_json_logging(enabled: bool) : bool
+enable_csv_logging(enabled: bool, filepath: str) : bool
+get_logging_state() : Dict[str, Any]
+set_csv_path(filepath: str) : bool
+get_available_collectors() : List[Dict[str, str]]
+get_current_collector() : Dict[str, Any]
+switch_collector(collector_type: str) : bool
+scan_controllers() : List[Dict[str, Any]]
+connect_controller(index: int) : bool
+set_mouse_tracking_mode(mode: str, direction_only: bool) : bool
+get_mouse_settings() : Dict[str, Any]
+set_mouse_settings(delta_smoothing: float, joystick_sensitivity: float, angle_grouping: int) : bool
+get_available_tracking_modes(direction_only: bool) : List[str]
+get_tracked_keys() : List[str]
+add_tracked_key(key: str) : bool
+remove_tracked_key(key: str) : bool
+toggle_button_tracking(button_id: str, track: bool) : bool
+toggle_edit_mode() : bool
+is_edit_mode() : bool
+toggle_fullscreen() : bool
+get_application_state() : Dict[str, Any]
«service»
InteractionApiIntegration
+get_interaction_api_integration() : InteractionApiIntegration
+_broadcast_event(event_dict: Dict) : void
+_handle_core_event(event_type: EventType, data: Any) : void
«static API»
InputDataAPI
+get_active_keys() : List[str]
+is_key_active(key: str) : bool
+get_mouse_position() : Tuple[int, int]
+get_mouse_button_state(button: str) : bool
+get_mouse_direction() : Optional[str]
+is_mouse_moving() : bool
+get_metadata() : Dict[str, Any]
+get_current_event() : Optional[InputData]
+get_key_events(count: int) : List[InputData]
+get_mouse_movement_events(count: int) : List[InputData]
+get_mouse_button_events(count: int) : List[InputData]
+get_controller_events(count: int) : List[InputData]
+is_controller_connected() : bool
+get_controller_name() : str
«rich library»
RichComponents
-Console: Class
+Table: Class
+Panel: Class
+Layout: Class
+Live: Class
+box: Module
«threading»
ThreadingComponents
+threading.Event: Class
+threading.Thread: Class
+threading.Lock: Class
+time.time() : float
+time.sleep(seconds: float) : void
«module»
CoreEventSystem
+EventType: Enum
+subscribe(event_type: EventType, handler: Callable) : void
+unsubscribe(event_type: EventType, handler: Callable) : void
+dispatch(event_type: EventType, data: Any) : void
InputData
+timestamp: float
+event_type: str
+key: Optional[str]
+active_keys: Optional[List[str]]
+rel_x: Optional[float]
+rel_y: Optional[float]
+button_pressed: Optional[str]
+to_dict() : Dict[str, Any]
«enumeration»
EventType
+INPUT: EventType
+STATE: EventType
+ERROR: EventType
uses
uses for output
uses for display
uses for rendering
creates display
uses for concurrency
uses for thread safety
creates and manages
uses for cross-platform key detection
uses for key listening
subscribes to events
receive events
delivers events
access
output
inputs
dynamic
defines